



Chapter 7

Integrity, Views, Security, and Catalogs



Integrity, Views, Security, and Catalogs



School of software Yunnan University

7.1 Integrity Constraints

7.2 Creating Views

7.3 Security: The Grant Statement in SQL

7.4 System Catalogs and Schemas



Integrity, Views, Security, and Catalogs



School of software Yunnan University

□ Introduction

- The DBA must begin by creating the tables and constraints and loading the data. Impose **integrity constraints** on the columns
- Then provide **views** of the data, virtually restructuring the physical tables into variant table forms, to simplify access to data.
- Provide **security**, so that only authorized users are able to read or update certain confidential data.
- The structure of the various tables, views, and other objects of a database are made available to the DBA through a set of system-defined tables, called **system catalogs**.



7.1 Integrity Constraints



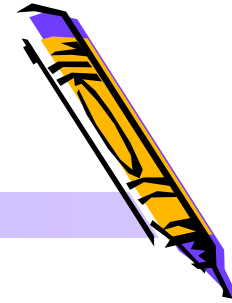
School of software Yunnan University

□ 数据完整性的类型

- 数据完整性类型包括：实体完整性、参照完整性和用户定义完整性
 - 实体完整性(Entity Integrity)：现实世界的实体是可区分的，即它们具有某种唯一性标识。相应地，关系模型中主键应作为唯一性标识。因此实体完整性规则规定基本关系的所有主键(Primary Key)都不能取空值(NULL)。



7.1 Integrity Constraints



School of software Yunnan University

□ 数据完整性的类型

- 数据完整性类型包括：实体完整性、参照完整性和用户定义完整性
 - 参照完整性（Referential Integrity）：参照完整性维护表与表之间的相关性，通常用“主键(Primary Key)/外键(Foreign Key)”保证，其中Foreign Key可以取NULL值，或取其参照表中Primary Key的取值。



7.1 Integrity Constraints



School of software Yunnan University

□ 数据完整性的类型

- 数据完整性类型包括：实体完整性、参照完整性和用户定义完整性
 - 用户定义的完整性(User_defined Integrity)：针对某一具体数据的约束条件，由应用环境决定。例如：某个属性必须取唯一值(UNIQUE)、某个非主属性不能取NULL值、某个属性的取值范围在0~100之间(CHECK)等等。



7.1 Integrity Constraints



School of software Yunnan University

□ 数据完整性

- 保证数据完整性（主要指数据的正确性与一致性）是数据库管理员的最重要任务之一。
 - 可以在数据表创建的时候通过约束(Constraint)、规则(Rule)和默认(Default)来保证数据的完整性
 - 或通过存储过程和触发器保证数据完整性
 - 也可以在应用程序层保证数据完整性（这是应用程序设计的问题）



7.1 Integrity Constraints



School of software Yunnan University

□ 保证数据完整性

■ 声明性数据完整性

声明性数据完整性用**约束(Constraint)**、**规则(Rule)**和**默认(Default)**在数据库中提供保证，这是保证完整性的较好方法。它驻留在数据库内，编程简单，开销小，能更集中管理和保证数据的一致性。

■ 过程性数据完整性

过程性数据完整性用**存储过程**、**触发器**和**应用程序代码**保证，通常较为复杂、开销较大，但可以执行更多的业务规则。

■ 通常，过程性数据完整性是声明性数据完整性的补充




```

CREATE TABLE [schema.]tablename
    ({columnname datatype [DEFAULT {default_constant|NULL}] [col_constr {col_constr...}]
    | table_constr}-- choice of either columnname-definition or table_constr
    {,{columnname datatype [DEFAULT {default_constant|NULL}] [col_constr {col_constr...}]
    | table_constr}
    ...});          -- zero or more additional columnname-def or table_constr

```

约束单个列的Col_constr形式如下:

```

{NOT NULL |          -- this is the first of a set of choices
[CONSTRAINT constraintname] -- if later choices used, optionally name constraint
    UNIQUE           -- the rest of the choices start here
    | PRIMARY KEY
    | CHECK (search_cond)
    | REFERENCES tablename [(columnname)]
    [ON DELETE CASCADE]}

```

约束多个列的table_constr形式如下:

```

[CONSTRAINT constraintname]
    {UNIQUE (columnname {, columnname...})          -- choose one of these clauses
    | PRIMARY KEY (columnname {, columnname...})
    | CHECK (search_condition)
    | FOREIGN KEY (columnname {, columnname...})    -- following is all one clause
    REFERENCES tablename [(columnname {, columnname...})]
    [ON DELETE CASCADE]}

```

7.1 Integrity Constraints



School of software Yunnan University

□ Definition 7.1.1 Clauses of the Create Table Command.

- **Create Table**命令先要给表命名，表名之后的圆括号中是用逗号分隔的列定义以及表约束。列定义和表约束可以以任何顺序出现。每个列定义包括列的名字、数据类型和可选的 **DEFAULT**子句。当**SQL**的**Insert**语句没有给出该列的值时，就用这个子句的缺省值。



7.1 Integrity Constraints



School of software Yunnan University

□ Definition 7.1.2 Column Constraint

- **NOT NULL** — If NOT NULL appears in a col_constr, then the DEFAULT clause cannot specify NULL;
- **CONSTRAINT** — allows us to specify a name for each constraint other than NOT NULL, so that we can later drop the constraint with an ALTER Table statement.
- **UNIQUE** — It can be specified even if NOT NULL is not, and the column is then constrained so that all non-null values in the table are unique, but multiple nulls can exist for this column.



7.1 Integrity Constraints



School of software Yunnan University

□ Definition 7.1.2 Column Constraint

- **PRIMARY KEY** — A column with the PRIMARY KEY column constraint is implicitly defined to be NOT NULL and UNIQUE. The UNIQUE clause and PRIMARY KEY clause cannot both be used for a column, although the PRIMARY KEY clause and the NOT NULL clause can be used together.
- **CHECK**



7.1 Integrity Constraints



School of software Yunnan University

□ Definition 7.1.2 Column Constraint

- **REFERENCES** — The optional **ON DELETE CASCADE** clause that followed the REFERENCES clause requires that when a row in the referenced table is deleted that is referenced by rows in the referencing table, then those rows in the referencing table are deleted.
 - If not ON DELETE clause appears, then such a delete of a referenced row will simply **fail to execute**.



7.1 Integrity Constraints



可定义为
**Primary
Key**

□ Definition 7.1.2 Column Constraint

use cap;

create table sales(sid char(4) not null unique,
cid char(4) **references** customers **on delete cascade**,
dollars real **constraint** dollars_min **check** (dollars>0.0)
);

定义约束名

该约束可以应用于不同的数据表中，且同一数据库中的约束不能同名

主键表为customers
可定义为
Customers (cid)



7.1 Integrity Constraints



School of software Yunnan University

□ Definition 7.1.3 Table Constraint

- **UNIQUE** — It is possible to specify a set of columns that must be unique in combination.
- **PRIMARY KEY** — Specify a non-empty set of columns to be a primary key. Every column that participates in PRIMARY KEY clause is implicitly defined to be NOT NULL.
- **CHECK**
- **FOREIGN KEY... REFERENCES...[ON DELETE [CASCADE]]**



7.1 Integrity Constraints



School of software Yunnan University

□ Example 7.1.2 CAP

```
create table orders (ordno integer not null, month char(3),  
cid char(4) not null, aid char(3) not null, pid char(3) not null,  
qty integer not null constraint qtyck check(qty >= 0),  
dollars float default 0.0 constraint dollarsck check(dollars >= 0.0),  
primary key (ordno),  
constraint cidref foreign key (cid) references customers,  
constraint aidref foreign key (aid) references agents,  
constraint pidref foreign key (pid) references products);
```



7.1 Integrity Constraints



School of software Yunnan University

□ Primary Keys, Foreign Keys, and Referential Integrity

■ Definition 7.1.4 Foreign Key, Referential Integrity.

如果表 **T 1** 中任意行的 **F** 值的组合，至少包含一个空值或与被引用表 **T 2** 中为候选键或主键的列集合 **P** 的组合值匹配，则列集合 **F** 被定义为外键。换句话说，如果表 **T 1** 的每一行中 **F** 的列都满足（1）至少有一列为空值（如果该列允许为空），（2）如果不包含空值，就必须与表 **T 2** 中某行的相应 **P** 的组合值相等，则参照完整性约束是有效的。



7.1 Integrity Constraints



School of software Yunnan University

□ Primary Keys, Foreign Keys, and Referential Integrity

	Insert	Delete	Update
主键 (被外键引用)	no test	test-1	test-2
外键 (引用主键)	test-3	no test	test-4

图7-4 参照完整性测试的表格



7.1 Integrity Constraints



School of software Yunnan University

□ Primary Keys, Foreign Keys, and Referential Integrity

	Insert	Delete	Update
主键(被外键引用)	无须测试	ON DELETE... 或缺省 NO ACTION	ON DELETE... 或 ON UPDATE... 或缺省 NO ACTION
外键	必要时语句失败	无须测试	必要时语句失败



SQL Language—数据完整性

School of software Yunnan University

□ 约束

- **级联参照完整性**：直到**SQL Server 2000**中才加入进来的功能，在**Create Table**与**Alter Table**命令中增加了**ON DELETE**和**ON UPDATE**子句。对这些子句指定**CASCADE**时，父表的操作级联到子表。

Alter Table Employee

Add **Constraint** emp_dept_FK **Foreign Key** (dept)

References department(dept_no)

ON DELETE CASCADE

ON UPDATE CASCADE

- 对**department**表中**dept_no**的任何改变都会影响到**employee**表。



SQL Language—数据完整性



School of software Yunnan University

□ 约束

- **Check**: 约束列中能输入的值, **Check**约束用布尔表达式实现, 当求出的值为**false**时, 则不能进行插入或更新操作。
- **Check**约束可以保证数据符合一定的格式, 还可以引用同一表中的列, 或引用函数。



SQL Language—数据完整性



School of software Yunnan University

□ 约束

■ Check:

Create Table inventory

(item_code char(4) NOT NULL constraint CK_inventory_item_code
Check (item_code like '[0-9][0-9][0-9][0-9]'),

high_volume int NOT NULL constraint CK_inventory_high_volume
Check (high_volume>0),

low_volume int NOT NULL constraint CK_inventory_low_volume
Check (low_volume>0),

Constraint CK_inventory_hi_lo_check

Check (high_volume>=low_volume)



SQL Language—数据完整性



School of software Yunnan University

□ 管理约束

- 管理约束包括收集约束信息、关闭与重新启用约束、删除约束。
- **收集约束信息**：可以查询信息结构图，也可以在查询分析器中调用系统存储过程**Sp_help**和**Sp_helpconstraint**收集信息。
- **关闭与重新启用约束**：可以关闭**Check**与**Foreign Key**约束，使之不检查现有数据或装入的新数据。
 - 在现有的表中添加约束时，如果只对新数据加约束，不检查已有数据，则采用**With NOCHECK**

Alter Table inventory

WITH NOCHECK

add UnitPrice Constraint CK_Product_UnitPrice Check
(UnitPrice>=0)



SQL Language—数据完整性



School of software Yunnan University

□ 管理约束

- **关闭与重新启用约束**：可以关闭**Check**与**Foreign Key**约束，使之不检查现有数据或装入的新数据。

- 如果要关闭已有约束，对加入新数据不用约束，则采用**NOCHECK**

Alter Table inventory

NOCHECK

Constraint CK_Product_UnitPrice

Alter Table inventory

NOCHECK

Constraint CK_Product_UnitPrice



SQL Language—数据完整性



School of software Yunnan University

□ 管理约束

- **删除约束**：可以用**Alter Table** 命令删除约束，也可以在企业管理器的设计表选项中删除

Alter Table Product

Drop Constraint CK_product_UnitPrice



SQL Language—数据完整性



School of software Yunnan University

□ 默认

- 默认可以指定列中不输入数值时的默认值，默认可以是求值为常量的任何项目，如常量、内置函数或数字表达式。
- 默认有两种：声明式默认和关联默认，其功能是相同的，只是实现方式不同。
- **声明式默认**：声明式默认只是另外一种约束，因此用**Create Table**和**Alter Table**命令实现。

- 在**Employee**表中，不指定电话号码时，默认数值为UNLISTED

Alter Table Employee

Add Constraint DF_emp_ph **Default** 'UNLISTED' **for** Phone

- 删除默认

Alter Table Employee

Drop Constraint DF_emp_ph



SQL Language—数据完整性



School of software Yunnan University

□ 默认

- **关联默认**：关联默认是独立于数据表创建的一个数据库对象，只有将其应用于一个数据表时，才对该表中对应的数据起作用。

- 创建：

Create Default *default_name*

AS *constant_expression*

- 只能在当前数据库中创建关联默认对象，其中 *constant_expression* 可以是常量、内置函数、算术表达式、全局变量等，但不能包含任何列名或其它数据库对象
- 在 *constant_expression* 中，字符和日期类型的数据应包含在单引号（ ' ' ）中，二进制数据必须以 **0x** 开头，货币类型数据必须以（ \$ ）开头。其它数据则无约束。



SQL Language—数据完整性



School of software Yunnan University

□ 默认

- **关联默认**：关联默认是独立于数据表创建的一个数据库对象，只有将其应用于一个数据表时，才对该表中对应的数据起作用。

- 创建：

Create Default *default_name*

AS *constant_expression*

- 在任何表中，不指定电话号码时，默认数值为UNLISTED

Create Default phone_df

AS 'UNLISTED'





SQL Language—数据完整性

School of software Yunnan University

□ 默认

■ **关联默认**：关联默认是独立于数据表创建的一个数据库对象，只有将其应用于一个数据表时，才对该表中对应的数据起作用。

- 应用：将一个默认对象应用于数据表需要使用系统存储过程

Sp_bindefault实现

Sp_bindefault [@defname=] 'default',

需要应用的默认对象名称

[@objname=] 'object_name'

该默认对象应用到的列名
“表名.列名”





SQL Language—数据完整性

School of software Yunnan University

□ 默认

■ 关联默认:

- 将电话号码的默认对象应用于**inventory**表中的**phone**列

```
sp_bindefault @defname='phone_df',  
              @objname='inventory.phone'
```

等价于:

```
sp_bindefault phone_df, 'inventory.phone'
```

- 可以将新的默认对象应用于已有默认对象的属性列，此时，旧的默认对象被自动删除
- 可以解除默认对象的绑定**sp_unbindefault()**
 - **Sp_unbindefault inventory.phone**
- 不能将默认对象应用于**timestamp**类型或**identity**属性列。





SQL Language—数据完整性

School of software Yunnan University

□ 默认

■ 关联默认:

- 默认的另一个用法是根据系统函数产生数值

Create Table Employee

(Emp_no int identity(1,1) constraint Emp_PK_emp_no Primary Key,

Lname char(20) Not Null,

Fname char(20) Not Null,

Phone char(13),

Dept smallint Not Null,

Uptime datetime Default Getdate() not null,

Updby varchar(30) Default Suser_Sname() not null

其中: Suser_Sname()返回登录标识名计算机名\用户名





SQL Language—数据完整性

School of software Yunnan University

□ 规则

- 规则用于约束列中可以存放的数值范围，对于一个具体的数据表而言，规则对象所执行的功能与**CHECK**约束类似，但规则是一种独立于数据表的数据库对象，只有将其应用于一个数据表时，才对相应的数据起作用
- 规则的使用与默认类似
- 例如：用规则限定数据格式

Create rule phone_rule

AS @phone Like '([0-9][0-9][0-9][0-9])-[0-9][0-9][0-9][0-9][0-9]
[0-9][0-9] '





SQL Language—数据完整性

School of software Yunnan University

□ 规则

■ 生成规则

Create RULE code_rule

AS @code in ('r01', 'r02', 'r03')

■ 使用系统存储过程**Sp_bindrule**将规则关联到列或用户定义数据类型

Sp_bindrule code_rule, 'inventory.item_code'

Sp_bindrule code_rule, **color**

■ 解除关联

Sp_unbindrule color

Sp_unbindrule inventory.item_code

用户定义数据类型



7.1 Integrity Constraints



School of software Yunnan University

□ Non-Procedural and Procedural Integrity Constraints: Triggers

- The SQL integrity clauses is non-procedural constraint
- The procedural integrity, known as *triggers*.
- We claim that non-procedural constraint miss a good deal of power that is available if constraints can be procedurally specified. However, there is a trade-off in complexity of representation that we need to pay for this added power.



7.2 Creating Views



School of software Yunnan University

□ Create Views

- A view is a table that results from a Subquery, but which has its own name and can be treated in most ways as if it were a base table.

■ Example 7.2.1

```
create view agentorders (ordno, month, cid, aid, pid, qty, charge, aname, acity, percent)
as select o.ordno, o.month, o.cid, o.aid, o.pid, o.qty, o.dollars,
       a.aname, a.city, a.percent
from orders o, agents a where o.aid = a.aid;
```



7.2 Creating Views



School of software Yunnan University

□ Create Views

- It is extremely important to realize that a view is a ***window*** on the data of the base tables.
- We are not taking a snapshot of the data at the time a view is created, but only storing definitions that must be interpreted with each new query.
- Queries on views are immediately responsive to changes in the underlying base table data.



7.2 Creating Views



School of software Yunnan University

□ 生成与管理视图

- 视图就是把查询存放成数据库对象，除了少数例外，视图返回与任何**Select**语句相同的数据，包括数据的子集（表中所选列）或父集（把两个或多个表用**join**或**union**操作符联接起来）。视图常用于返回累计值。视图是一个虚表，因此一旦基本表中的数据发生变化，从视图查询出的数据也就随之改变了。
- 视图是采用Select语句来构造的，在定义一个视图时，只是把其定义存放在系统的数据字典中，直到用户使用视图时才去求对应的数据。视图一经定义，就可以和基本表一样被查询、被删除，也可以在一个视图之上再定义新的视图。但对视图的更新操作则有一定的约束。



SQL Language—视图



School of software Yunnan University

□ 创建视图 “Create View”

■ 格式:

CREATE VIEW <视图名>[(<列名>[, <列名>]...)]

AS <子查询>

[WITH CHECK OPTION];

- 其中子查询可以是任意的**SELECT**语句，但通常不允许含有**ORDER BY**、**COMPUTE**子句和**DISTINCT**短语。
- 视图不允许参照到一个临时表
- **WITH CHECK OPTION**表示视图进行**UPDATE**，**INSERT**和**DELETE**操作时要保证更新、插入或删除的行满足视图定义的谓词条件（即子查询中的条件表达式）。



SQL Language—视图



School of software Yunnan University

□ 创建视图 “Create View”

■ 例1：使用基本表的列名作为视图的列名

- Create View view1

AS

Select s#, sname

From S

■ 例2：从多个数据表创建视图

- Create View view2

AS

Select s.s#, sname, c#, grade

From S, SC

where S.s#=SC.s#



SQL Language—视图



School of software Yunnan University

□ 创建视图 “Create View”

■ 例3：使用SQL Server函数创建视图

- 查询每个学生选修课程的门数和平均成绩
- create View S_grade(S#, C_num, AVG_grade)

AS

Select S#, COUNT(C#), AVG(grade)

From SC

Group by S#

create view or function failed because no column name was specified for column 2,3.



7.2 Creating Views



School of software Yunnan University

□ With CHECK OPTION

- The optional WITH CHECK OPTION clause specifies that **Insert** and **Updates** performed through the view to result in base table changes should not be permitted if they result in rows that would be invisible to the view Subquery;



7.2 Creating Views



School of software Yunnan University

□ With CHECK OPTION

- use cap;
- go
- create view custs as select * from customers
where discnt<=15.0
with check option

1、视图custs仅显示customers表中满足where条件的行

2、视图数据的更新必须满足视图上定义的约束条件，如果不满足则不会执行

- update custs set discnt=discnt+4.0

3、如果满足，其基本表customers表将所有满足定义的数据进行更新，但只有满足custs定义的数据才在视图中显示



7.2 Creating Views



School of software Yunnan University

□ Updateable and Read-Only Views

- Insert, Update, and Delete operations are permitted for updatable views and not permitted for read-only views.
 1. The FROM clause of the subquery must contain only a single table, and if that table is a view table it must also be an updatable view table.
 2. Neither the GROUP BY nor the HAVING clause is present.
 3. The DISTINCT keyword is not specified.
 4. The WHERE clause does not contain a subquery that references any table in the FROM clause, directly or indirectly via views.
 5. All result columns of the subquery are simple column names (that is, there are no arithmetic expressions such as `avg(qty)` or `qty+100`), and no column name appears more than once in distinct result columns.



7.2 Creating Views



School of software Yunnan University

□ Updateable and Read-Only Views

■ [7.2.6]

- Create view colocated
as select cid, cname, aid, aname, a.city as acity
from customers c, agents a
where c.city=a.city

违反了规则1

	cid	cname	aid	aname	acity
1	c001	TipTop	a05	Otasi	Duluth
2	c002	Basics	a06	Smith	Dallas
3	c003	Allied	a06	Smith	Dallas
4	c004	ACME	a05	Otasi	Duluth



7.2 Creating Views



School of software Yunnan University

□ Updateable and Read-Only Views

■ [7.2.7]

- Create view agentsales(aid, totalsales)
as select aid, sum(dollars)
from orders
group by aid

违反了规则2

	aid	totalsales
1	a01	1400.0000
2	a02	180.0000
3	a03	4228.0000
4	a04	450.0000
5	a05	2144.0000
6	a06	1400.0000
7	a07	450.0000



SQL Language—视图



School of software Yunnan University

□ 删除视图 “Drop View”

■ 格式:

Drop View <视图名>

- 视图删除后视图的定义将从数据字典中删除。但是用于定义该视图的基本表不会改变，由该视图导出的其他视图定义仍在数据字典中，不过该视图已失效，用户使用时会出错，要用**DROP VIEW**语句将它们一一删除。

- 例如：删除视图**S_grade**

Drop View S_grade



7.2 Creating Views



School of software Yunnan University

□ The Value of Views

- Views add a security aspect to allow different users to see the same data in different ways.
- Views provide a way to make complex, commonly issued queries easier to compose.
- Views allow obsolete tables, and programs that reference them, to survive reorganization.



7.2 Creating Views

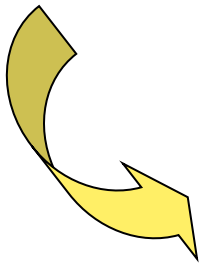


School of software Yunnan University

□ Create Views

■ Example 7.2.2

```
select sum(charge) from agentorders where acity = 'Toledo';
```



```
select sum(o.dollars) from orders o, agents a  
where o.aid = a.aid and a.city = 'Toledo';
```


7.2 Creating Views



School of software Yunnan University

□ 视图的作用

- 视图能够简化用户的操作——简化数据查询和处理操作，对多个表的联接查询简化为对一个视图的操作。
- 视图使用户能以多种角度看待同一数据——看到用户需要的数据，并且可以看到对数据的一些汇总、计算结果。
- 视图对重构数据库提供了一定程度的逻辑独立性——简化数据的交换操作
- 视图能够对机密数据提供安全保护——仅让用户看到一部分数据



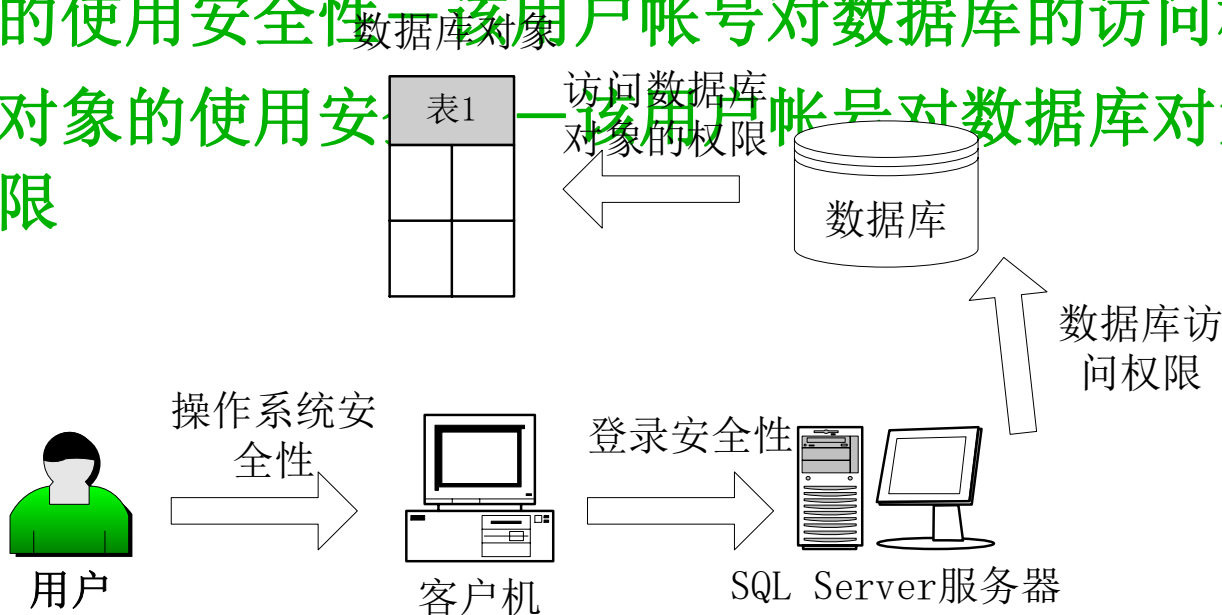
SQL Server的安全性机制



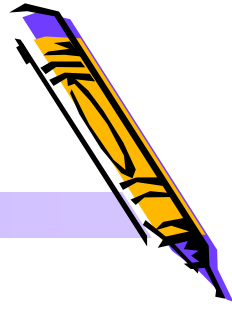
School of software Yunnan University

□ 目前，SQL Server的安全性机制主要划分为4个等级

- 客户机操作系统的安全性
- **SQL Server**的登录安全性—登录帐号和密码
- 数据库的使用安全性—该用户帐号对数据库的访问权限
- 数据库对象的使用安全性—该用户帐号对数据库对象的访问权限



SQL Server的安全性机制



School of software Yunnan University

□ 安全模式

■ 为了**SQL Server**服务器和数据库的安全，系统管理员应该规划一个高效的安全模式，一个高效的安全模式主要包括以下内容：

- 做一些细致的、具有前瞻性的安全规划
- 选择安全形式
- 配置安全角色
- 指定对象以及语句的许可权限



SQL Server的安全性机制



School of software Yunnan University

□ 安全模式

1. 安全规划（在系统开始运行之前完成）

- 确定采用何种登录验证方式
- 决定哪些用户将执行管理SQL Server服务器系统的任务，并为这些用户分配适当的服务器角色
- 决定哪些用户存取哪些数据库，并为这些用户添加到适当的数据库角色
- 给适当的用户或角色授予适当的存取数据库对象的权限，以使用户能够操作相应的数据库对象



SQL Server的安全性机制



School of software Yunnan University

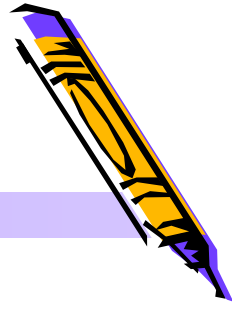
□ 安全模式

■ 为了**SQL Server**服务器和数据库的安全，系统管理员应该规划一个高效的安全模式，一个高效的安全模式主要包括以下内容：

- 做一些细致的、具有前瞻性的安全规划
- 选择安全形式
- 配置安全角色
- 指定对象以及语句的许可权限



SQL Server的安全性机制



School of software Yunnan University

□ 安全模式

2. 选择安全形式（主要指用户登录的验证方式）

- **Windows验证方式**：完全采用**Windows**服务器的验证，只要可以登录到**Windows**的用户，就可以登录到**SQL Server**数据库系统
- **Windows与SQL Server混合验证方式**：更灵活，不能登录到**Windows**的用户，只要是**SQL Server**的用户就可以登录到数据库系统



SQL Server的安全性机制



School of software Yunnan University

□ 安全模式

■ 为了**SQL Server**服务器和数据库的安全，系统管理员应该规划一个高效的安全模式，一个高效的安全模式主要包括以下内容：

- 做一些细致的、具有前瞻性的安全规划
- 选择安全形式
- 配置安全角色
- 指定对象以及语句的许可权限



SQL Server的安全性机制



School of software Yunnan University

□ 安全模式

3. 配置安全角色

- **服务器角色**：服务器角色允许登录到**SQL Server**服务器，并具有操作服务器的权限。
- **数据库角色**：数据库角色允许对数据库对象进行操作。
- 创建用户后，为用户分配适当的角色

4. 指定对象及语句许可权限

- 授予用户对数据库中具体对象的操作权限和对**SQL**语句的使用权限
- 数据库对象（如数据表、视图、存储过程等）
- 任何对数据表或视图数据的读取（**Select**）、插入（**Insert**）、更新（**Update**）和删除（**Delete**）权限需要明确授权。



SQL Server安全性一角色



School of software Yunnan University

□ 角色是管理SQL Server服务器操作权限和SQL Server数据库访问权限的单位。其成员继承角色所拥有的权限。

- 角色类似用户组，设置角色的目的就是为了方便数据库的管理
- 角色管理过程：
 - 创建角色
 - 将访问许可权限集中授予角色
 - 将需要拥有这些访问许可权限的用户加入到角色中



SQL Server安全性一角色



School of software Yunnan University

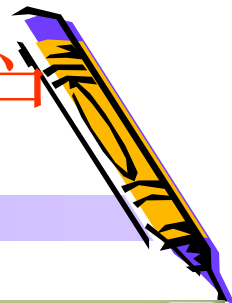
□ 固有角色

- 固有角色就是**SQL Server**预先定义好的系统角色，用户不能修改这些角色的任何属性
- **服务器角色**：（**8**个固有服务器角色）用户 **不能删除、不能修改**服务器固有角色，也 **不能创建**新的服务器固有角色
 - **SQL Server**服务器只有固有角色，不可以创建新的角色
- **数据库角色**：（系统为每个数据库设置了**10**个数据库固有角色）数据库角色在数据库的安全级别上创建，并被 **不能删除**

public是一个特殊的数据库角色，每个数据库都具有这一角色。数据库的所有用户都自动属于**public**角色，并且不能从该角色中删除。



SQL Server安全性—数据库用户



School of software Yunnan University

SQL Server 登录属性 — 新建登录

常规 | 服务器角色 | 数据库访问

名称(N): user1

身份验证

☐ Windows 身份验证(W)

域(M):

安全性访问:

☒ 允许访问(A)

☐ 拒绝访问(D)

☒ SQL Server 身份验证(S)

密码(P):

默认设置

指定此登录的默认语言和数据库。

数据库(D): master

语言(L): <默认>

确定 取消 帮助

SQL Server 登录属性 — 新建登录

常规 | 服务器角色 | 数据库访问

指定此登录可以访问的数据库(S)

许可	数据库	用户
☒	CAP	
☐	North...	
☐	master	
☐	model	
☐	msdb	
☐	pubs	
☐	student	

"CAP"的数据库角色(D):

数据库角色中允许

☒	public
☐	db_owner
☐	db_accessadmin
☐	db_securityadmin
☐	db_ddladmin

属性(P)

确定 取消 帮助

7.3 Security: The Grant Statement in SQL



School of software Yunnan University

□ Security

- The Grant statement is an SQL command issued by the owner of a table (base table or view table) to authorize various kinds of access (select, update, delete, or insert) to the table by another user or class of users.

- [7.3.1]

`grant select on customers to user1;`



7.3 Security: The Grant Statement in SQL



School of software Yunnan University

```
select * from customers
```

服务器: 消息 229, 级别 14, 状态 5, 行 1

拒绝了对对象 'CUSTOMERS' (数据库 'CAP', 所有者 'dbo') 的 SELECT 权限。



7.3 Security: The Grant Statement in SQL



School of software Yunnan University

```
grant select on customers to user1
```

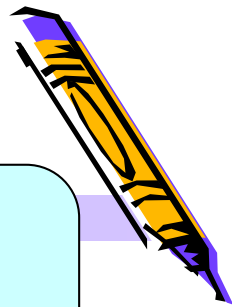
查询 — VALOR.CAP.user1 — 无标题4*

```
select * from customers
```

	cid	cname	city	discnt	
1	c001	TipTop	Duluth	12.00	
2	c002	Basics	Dallas	14.00	
3	c003	Allied	Dallas	10.00	
4	c004	ACME	Duluth	10.00	
5	c006	ACME	Kyoto	2.00	
6	c009	ii	china	3.00	



7.3 Security: The Grant Statement in SQL



School of software Yunnan University

The owner of a table automatically has all privileges

包括: SELECT, DELETE,
INSERT
UPDATE[(col {,col...})]
REFERENCES[(col {,col...})]

```
GRANT {ALL PRIVILEGES | privilege {, privilege...}}  
ON [TABLE] tablename | viewname  
TO {PUBLIC | user-name {, user-name...} } [WITH GRANT OPTION]
```

图7-16 基本SQL的Grant语句形式

用户具有了将得到的权限
授予其他用户的权限



7.3 Security: The Grant Statement in SQL



School of software Yunnan University

- Grant permission to user user1 on the customers table to insert or delete any row, update only the cname and city columns.
 - grant insert,delete on customers to user1
 - grant select, update(cname, city) on custview to user1



7.3 Security: The Grant Statement in SQL



School of software Yunnan University

- Grant permission to user user1 to perform all accesses on agents with percent greater than 5.
 - create view agentview as select * from agents where percent>5
 - grant all privileges on agentview to user1



7.3 Security: The Grant Statement in SQL



School of software Yunnan University

□ Revoke privileges on a table

```
REVOKE {ALL PRIVILEGES | privilege {, privilege...} }  
ON tablename | viewname  
FROM {PUBLIC | user-name {, user-name...} }  
{CASCADE | RESTRICT};      -- one of these is required in X/Open
```



7.4 System Catalogs and Schemas



School of software Yunnan University

- All relational database systems maintain system catalogs, tables (or views) maintained by the system that contain information about objects defined in the database.
- Such objects include tables defined with the Create Table, columns, indexes, views, privileges, constraints, and so on.
- DB2 UDB refers to catalog tables.
- The catalog tables are created at the time that a database is created
- The user is not supposed to update catalog tables directly.



7.4 System Catalogs and Schemas



School of software Yunnan University

□ Schemas

- The full name of a table within a database catalog is schemaname.tablename (where the schemaname is usually a user name).
- This naming system allows different users (such as student in a class) to employ the same tablename without ambiguity in the full database.
- In some database systems, besides a schema for each user, there are additional schemas used, such as schema SYSCAT in DB2 UDB.

